

layout	default
parent	Frontend
title	Lab 3 - Binder
nav_order	4

Lab 3 - Binder

In this lab we will write a Binder for our Tiger compiler. The Binder is responsible for the following tasks:

- Wrapping the program inside a top-level `main` function;
- Binding each identifier (function call or variable) to its declaration;
- Decorating each identifier and each declaration with its depth;
- Marking escaping declarations as such;
- Decorating each break with its parent loop.

The Binder must also raise an error when:

- An identifier is used but not declared;
- An identifier is declared twice in the same scope;
- A break is used outside of a loop or inside a Let declaration (except if enclosed by a loop).

Setup

Retrieve the lab code and commit it to your git with the following commands,

```
$ mkdir lab3
$ cd lab3
$ wget -qO- www.sifflez.org/lectures/compil/lab3/dragon-tiger.tar.gz | tar zxv
$ git add -f dragon-tiger
$ git commit -m "Import dragon-tiger for lab3" dragon-tiger
```

The `-f` for `git add` is necessary to add the (usually ignored) `src/parser/libparser.a` which comes in the archive.

Now let us build the project,

```
$ cd dragon-tiger
$ ./configure
$ make
```

If everything worked as expected, you should be able to run the compiler driver `dtiger` as follows,

```
$ src/driver/dtiger --help
Options:
  -h [ --help ]           describe arguments
  --dump-ast              dump the parsed AST
  -b [ --bind ]           run the binder on the parsed AST
  --trace-parser          enable parser traces
  --trace-lexer           enable lexer traces
  -v [ --verbose ]        be verbose
  --input-file arg        input Tiger file

$ echo "a * b * c * d" > test.tig
$ src/driver/dtiger -v -b --dump-ast test.tig
function main(): int =
(
  (((a*b)*c)*d);
  0
)
```

Ensure that you test thoroughly and commit each feature. Follow precisely the instructions, this lab is graded and machine corrected!

Important note

Some of you may not have completed the previous assignment. To this effect, the current archive contains a pre-built parser library (so as not to give you the full solution) and this lab may be done separately.

However, when you have completed the previous assignment, feel free to replace the `src/parser/` directory by a copy of the one from your previous assignment. This will be most satisfactory, as your code from the first step will be used also in the second step. Do not forget to add `parser` to the `SUBDIRS` variable in `src/Makefile.am` (it must be the **first** subdirectory there because it auto-generates files used in other subdirectories) and `src/parser/Makefile` in `configure.ac`.

You might also want to propagate your evaluator code although this will not be needed.

Code organization

The binder is called with the `-b` option. It is interesting to also add the options `-v --ast-dump` which provide a verbose AST dump. The verbose AST dump shows the depth and bound declaration for every decorated identifier. Because the Binder is not yet operational, the verbose AST dump does not yet show anything relevant.

A starting stub is provided for the Binder and it can be found at `src/ast/binder.[hh,cc]`.

► Read carefully the provided code and ensure that you understand which method is wrapping the program inside a main function as shown below,

```
$ echo "a * b * c * d" | src/driver/dtiger -v -b --dump-ast -
function main(): int =
(
  (((a*b)*c)*d);
  0
)
```

► Why is it necessary to call the function `enter_primitive` in the Binder's constructor? Do you understand how they are not part of the tree, but they can be referenced by it?

► Read and make sure you understand the methods `current_scope`, `push_scope`, `pop_scope`, `enter` and `find`.

Binding Variable Declarations

Your first task is binding each variable usage to its declaration.

For example, the expected output for the simple program

```
let var a := 0 in a end
```

is,

```
$ echo "let var a := 0 in a end" | src/driver/dtiger -b --dump-ast -v -
function main(): int =
(
  let
    var a := 0
  in
    a/*dec1:1.5-7*/
  end;
  0
)
```

As you can see, the usage of `a` indicates that it refers to the variable declared at line 1 columns 5-7 (location points to the `var` keyword).

Currently, your binder does nothing.

► Add support for binding each variable usage to its declaration. To bind a variable identifier `id` to its declaration `dec1` you shall use the method `id.set_dec1(dec1)`. Your binder must adhere to the *scoping rules* of Tiger.

► Ensure you detect the following illegal cases:

- An identifier is used but not declared
- An identifier is declared twice in the same scope

In those illegal cases, you must raise a fatal error with the `utils::error()` function.

Ensure that your Binder works on complex scenarios with hiding or escaping, including with loop indices who need their own scope and cannot be assigned to.

Because function arguments are handled as `VarDec1` nodes in the AST, they should also be properly bound by your code.

Binding Function Declarations

► Add support for binding each function call to its declaration. Remember that in Tiger, a set of consecutive function declarations can be mutually referencing.

For example the following Tiger program,

```
let
  function even(n : int) : int = if n = 0 then 1 else odd(n-1)
  function odd(n : int) : int = if n = 0 then 0 else even(n-1)
in
  print_int(odd(5))
end
```

will produce the following verbose AST dump,

```
function main(): int =
(
  let
    function even/*main.even*/(n: int) =
      if
        (n/*decl:2.17*/=0)
      then
        1
      else
        odd/*decl:3.3-10*/((n/*decl:2.17*/-1))
    function odd/*main.odd*/(n: int) =
      if
        (n/*decl:3.16*/=0)
      then
        0
      else
        even/*decl:2.3-10*/((n/*decl:3.16*/-1))
  in
    print_int/*decl:<none>:0.0*/(odd/*decl:3.3-10*/(5))
  end;
  0
)
```

Declarations nodes for the primitive functions (such as `print_int`) have already been inserted in the top-level scope. They require no special handling. In the verbose AST dump, they are marked with a no-location `/*decl:<none>:0.0*/`.

► Make sure that the error handling code also works for function calls and declarations.

Remember to test your implementation in corner cases and complex scenarios.

Computing the depth of nodes and marking escaping declarations.

To handle escapes later on, we need to compute the depth of variable, function declarations, identifiers and function calls.

The depth is the number of nesting function levels. For example a top-level declaration has depth 0. A declaration that lives inside a top-level function has depth 1. A declaration inside a doubly nested function has depth 2.

► Implement depth computation and decoration in your Binder. To decorate a node `n` with its depth `d`, you shall use `n.set_depth(d)` ;

► All variables declarations that are accessed from a different depth should be marked as *escaping*. To mark a declaration `d` as escaping, you shall use `d.set_escapes()` .

The AST verbose dump shows non-null depth differences between a declaration and its use; it also shows escaping variables with `/*e*/` .

For example, the following Tiger program,

```
let var a := 0
  function f() : int = a
in f() end
```

should produce the following verbose dump,

```
function main(): int =
(
  let
    var a/*e*/ := 0
```

```

function f/*main.f*/(): int =
  a/*decl:1.5-7 depth_diff:1*/
in
  f/*decl:2.5-12*/()
end;
0
)

```

As you can see the depth difference between `a` and its declaration is one; therefore the declaration of `a` escapes.

Breaks and loops

Each `break` statement should be decorated with its parent loop. To do so you shall use the function `break.set_loop(loop)`; where `loop` is of class `Loop` (either a `ForLoop` or a `WhileLoop`).

► Decorate each `break` with its parent loop. To achieve this you may use a class member variable to record the visited loops.

► Ensure that you raise an error if a `break` is used outside of a loop or directly inside the declaration part of a `Let` block.

For example, the tiger program `while(1) do break` emits the following verbose AST dump,

```

function main(): int =
(
  while (
    1
  ) do
    break/*loop:1.1-5*/;
  0
)

```